C++ Simulation and Performance Comparison of CPU Scheduling Algorithms

Ameer Khan A *School of Computer Science and Engineering*
VIT Vellore
Email: ameerkhan.a2024@vitstudent.ac.in

Registration No: 24BEL0034

Source Code: https://github.com/ameerkhan-vitv/CPU-Scheduling-Simulation-Results

***Abstract*—This paper presents a performance comparison of five standard CPU scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF), Priority, Round Robin (RR), and a Multi-Level Feedback Queue (MLFQ). We developed a discrete-event simulator in C++ to analyze how these algorithms handle different workloads. The test cases scale from 100 to 2,000 processes with randomized arrival times and burst times. We evaluate the algorithms based on average waiting time, average turnaround time, and total context switches. The results confirm that SJF minimizes waiting times for fixed loads, while Round Robin creates a clear trade-off between scheduling fairness and context-switching overhead.**

***Index Terms*—CPU Scheduling, C++, Simulation, Context Switching, MLFQ.**

I. Introduction

Operating systems use CPU scheduling to manage how processes share the processor. Choosing or configuring a scheduling algorithm is critical for system performance, as a poor setup can lead to process starvation or excessive overhead under heavy workloads.

In this project, we built a modular simulator in C++ to analyze five core scheduling methods under varying process loads. By capturing runtime data like turnaround times and waiting latencies across large datasets, we evaluate how well these textbook algorithms perform under different traffic volumes.

II. Simulator Implementation

The simulation engine is written in C++ and split into multiple files for clean organization: `process.h` defines the process attributes, `schedulers.h` holds function signatures, and `schedulers.cpp` handles the actual algorithm logic.

A. Workload Setup

The workload generator generates random test workloads ranging from 100 to 2,000 processes using regular distributions:

- Arrival Times: Randomly chosen between 0 and 20 time units.
- Burst Times (Execution requirements): Ranging from 2 to 12 units.
- Priorities: Assigned random tiers from 1 (highest) to 4 (lowest).

B. Implemented Algorithms

1) *FCFS:* Processes run in the exact order they arrive using a basic queue structure.
2) *SJF:* A non-preemptive algorithm that searches the ready queue and executes the job with the shortest execution time next.
3) *Priority:* Dispatches jobs based on their priority level, executing higher priority tiers first.
4) *Round Robin (RR):* Each process runs for a small time quantum. If it does not finish, it goes back to the end of the queue.
5) *MLFQ:* A two-queue system. New jobs go into Queue 1 with a quick time quantum ($Q = 3$). Incomplete jobs drop to Queue 2, which processes them using FCFS rules to keep the system balanced.

C. Data Structures and Memory Setup

To simulate operating system behavior correctly without slowing down the data sweeps, we utilized the C++ Standard Template Library (STL). Individual tasks are held inside a dynamic array framework (`std::vector`), allowing the workload generator to

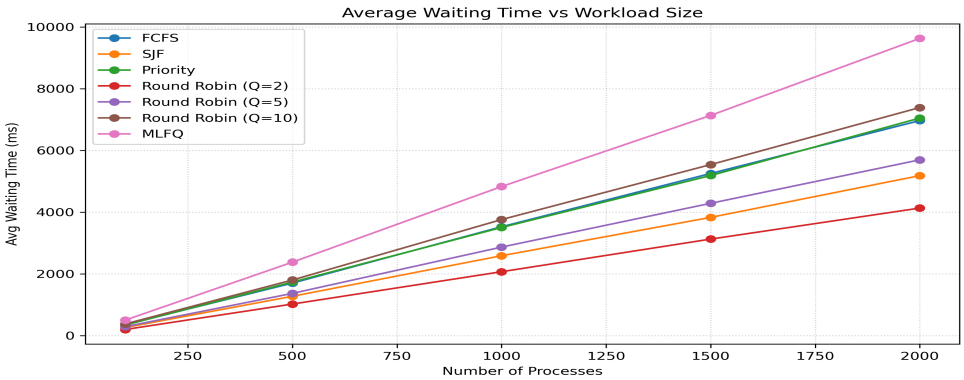


Fig. 1. Average Waiting Time vs Workload Size across scaling operational volumes.

initialize properties without compile-time bounds restrictions.

For scheduling tracking loops like Round Robin and the Multi-Level Feedback Queue, we managed active contexts using standard queue frameworks (`std::queue`). When new processes reach their designated system clock arrival boundaries, they are appended to the back of the pipeline.

The Shortest Job First simulator handles processing decisions by invoking stable array sorting mechanics (`std::sort`), ensuring that whenever the processor core hits an idle cycle, it scans the ready boundary indicators to isolate the index holding the smallest residual execution requirement.

III. Results and Performance Analysis

To draw clear conclusions for the project report, we evaluated each scheduling run across three concrete computational benchmarks:

- **Waiting Time:** The sum of periods a thread spent stuck inside the ready state line waiting for allocation.
- **Turnaround Time:** The absolute time gap tracking from a process's system introduction tick up to its completion cycle.
- **Context Switches:** The metric tracking how many times the processor had to interrupt a running task to swap execution focuses.

A. Workload Scaling Analysis

Our testing across different workload sizes highlights how process density affects waiting times. As shown in Fig. 1, SJF consistently maintains the lowest waiting times as the number of processes increases. MLFQ shows a higher average waiting time curve because it spends extra time context-switching and moving processes between different queues to ensure execution fairness.

B. Fixed-Load Comparison

When analyzing a steady load of 1,000 processes (Fig. 2), FCFS and Priority algorithms show higher average latencies. This is due to the convoy effect, where longer processes block later arrivals. SJF handles this load more efficiently by clearing short jobs out of the queue first. The metrics highlight that FCFS spikes significantly under concentrated traffic volume because it lacks any mechanism to rearrange resource queues based on job sizes.

C. Round Robin Context Switches

We tracked how the choice of time quantum in Round Robin affects system overhead. As shown in Fig. 3, a small quantum ($Q = 2$) causes a high number of context switches (over 2,100), meaning the CPU spends too much time swapping tasks. Increasing the quantum to $Q = 10$ drops the switches down near 1,150, proving that a larger quantum minimizes processing overhead.

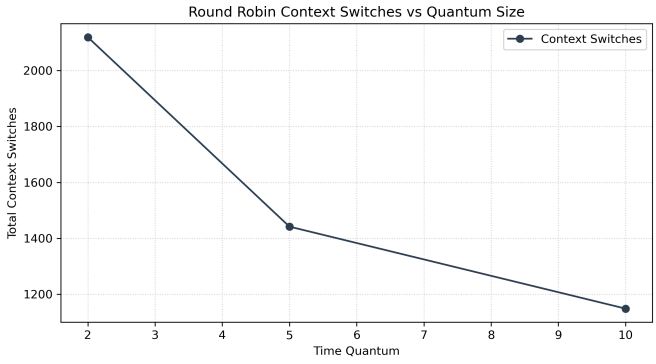


Fig. 3. Round Robin Context Switches vs Quantum Size.

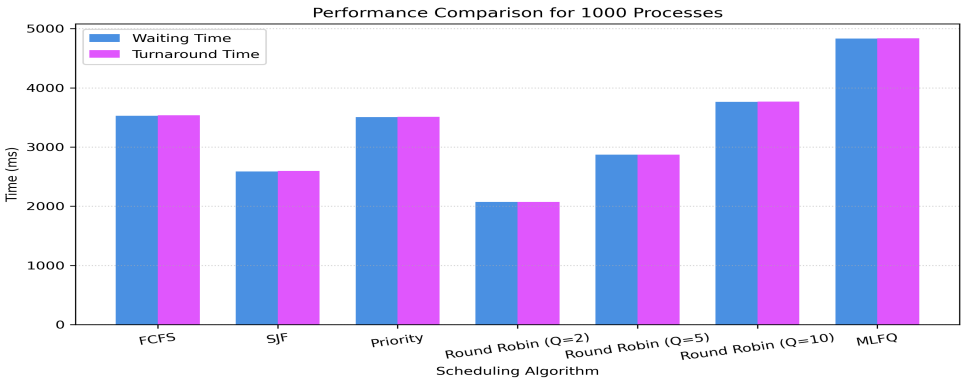


Fig. 2. Performance Comparison Metrics for a baseline load of 1,000 Processes.

IV. Discussion

The data highlights clear engineering trade-offs. SJF works best for minimizing latency, but it requires knowing a process's exact runtime beforehand, which is rarely possible in practical operating systems. Priority allocation provides fixed scheduling choices but risks creating execution bottlenecks if low-priority threads are permanently blocked by upcoming high-priority bursts.

Round Robin addresses fairness constraints cleanly but requires careful balance; if the time slice boundary drops too low, execution performance degrades into a loop of register swapping, whereas inflating the quantum converts the model back into a standard FCFS pipeline. The MLFQ implementation solves these issues by adjusting to process behavior at runtime, balancing scheduling fairness with real-world system responsiveness.

By moving tasks that consume excessive time slices down into lower-priority FIFO backfalls, our customized simulator proves that an adaptive queue can successfully shield highly interactive tasks from processing stalls caused by massive background CPU chunks.

V. Conclusion

We successfully built a modular C++ simulator to test and compare scheduling algorithms under load. The results closely follow theoretical models. For future upgrades, we plan to implement multi-core processor simulations and adaptive time quantum adjustments.

References

[1] N. Goel and R. B. Garg, "A Comparative Study of CPU Scheduling Algorithms," *arXiv preprint arXiv:1307.4165*, 2013.

[2] H. B. Parekh and S. Chaudhari, "Improved Round Robin CPU scheduling algorithm: Round Robin, Shortest Job First and priority algorithm coupled to increase throughput," *IEEE Global Trends in Signal Processing*, pp. 184–187, 2017.

[3] J. Omar, J. A. Omar, and S. R. M. Zeebaree, "Comparative analysis of the essential CPU scheduling algorithms," *Bulletin of Electrical Engineering and Informatics*, vol. 10, no. 5, pp. 2812–2820, 2021.

[4] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2015.

[5] P. Brucker, *Scheduling Algorithms*, 5th ed. Berlin, Germany: Springer-Verlag, 2007.